

Briefline: An End-to-End Framework for Multi-Task News Processing and Agentic Verification

Zhixiang Lin

Abstract—Processing long news articles requires multiple distinct tasks to be performed while ensuring that model-generated content is reliably grounded in the source text. To address redundant training data for multi-task models, difficulty in coordinating different tasks, and generated outputs that may deviate from news facts, this paper presents Briefline, which combines representative data selection, multi-task model fine-tuning, and evidence-based post-generation verification, and provides related recommendations through news retrieval and reranking. The system selects representative training samples from a large corpus of historical news, enables a single model to perform multiple news-processing tasks jointly, and applies the trained model to recent news. During application, the system reads the full source article to verify and revise questionable outputs. Experimental results show that Briefline reduces the training corpus by 86.83%; increases the average score across all tasks from 0.7178 to 0.8164 on the held-out test set; and reduces the derived hallucination score for news summaries from 0.0834 to 0.0490 after verification, a relative reduction of 41.2%. These results indicate that representative data selection and multi-task training can improve news-processing capability with a smaller training set, while conditional evidence-based verification can further improve the factual faithfulness of generated summaries.

Index Terms—semantic data selection; multi-task fine-tuning; evidence retrieval; agentic verification

I. INTRODUCTION

Processing long news articles typically involves multiple tasks, including summarization, topic classification, and keyphrase extraction. Conventional approaches often train or deploy a separate model for each task, which not only increases training and inference costs but also makes data processing, model maintenance, and system integration more complex. At the same time, generative models may produce content that is not supported by the source article, and a single generation step cannot ensure factual reliability. The core problem addressed in this project is how to perform multiple news-understanding tasks with lower system complexity while automatically checking and correcting the generated outputs.

To address this problem, Briefline uses joint fine-tuning to enable one model to perform all three tasks and builds an agentic workflow based on evidence retrieval, an LLM judge, and selective correction. The system applies the trained model to Guardian articles whose source and time range differ from those of the training corpus, creating a complete application workflow from news collection, content generation, evidence retrieval, and output verification to frontend presentation.

Experimental results show that semantic data selection reduces the training corpus by 86.83%; the multi-task model selected on the validation set improves the average score across all tasks by 13.74% on the held-out test set; agentic verification

reduces the RAGAS-derived hallucination score by 41.2%; and vLLM provides more than a threefold end-to-end evaluation speedup over Hugging Face Transformers.

II. RELATED WORK

A. Multi-Task Fine-Tuning

Multi-task instruction fine-tuning represents different tasks as a unified set of natural-language instructions and target outputs, enabling a single model to learn multiple capabilities in one training process. In the FLAN study, Chung et al. [1] systematically examined the effects of the number of fine-tuning tasks and model scale. Their results show that instruction fine-tuning on multiple tasks can improve overall model performance and generalization to unseen tasks. To reduce model adaptation costs, Zhang et al. [2] proposed AdaLoRA, which dynamically allocates the budget for low-rank updates according to parameter importance and provides a more flexible method for parameter-efficient fine-tuning of large language models.

B. Semantic Data Selection

Large training corpora may contain samples that differ in wording but are semantically similar, creating data redundancy. HNSW, proposed by Malkov and Yashunin [3], uses a hierarchical proximity graph for efficient approximate nearest-neighbor search. After text is represented as vectors, the algorithm can rapidly find similar samples. The Leiden algorithm, proposed by Traag et al. [4], is used for graph community detection and can produce internally connected communities. SemDeDup, proposed by Abbas et al. [5], uses embeddings generated by a pretrained model to identify semantic duplicates. Their experimental results show that removing some semantically duplicated data from the studied datasets can shorten training time while largely preserving model performance.

C. Retrieval and Automated Evaluation

RAG, proposed by Lewis et al. [6], combines a parametric language model with a retrievable non-parametric knowledge source, allowing generation to use information beyond the model parameters. ColBERT, proposed by Khatib and Zaharia [7], encodes queries and documents separately and then uses late interaction to compute fine-grained relevance, balancing retrieval quality and computational efficiency.

Separately from retrieval research, another line of work explores the use of language models to evaluate generated outputs automatically. Zheng et al. [8] systematically studied LLM-as-a-Judge and showed that strong language models can

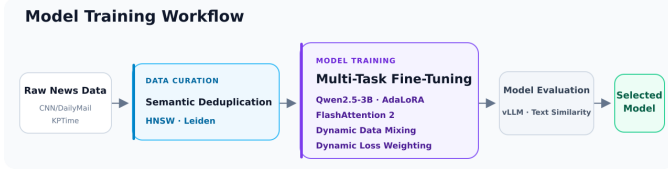


Fig. 1. Model training workflow.

evaluate open-ended outputs, while also noting that the method may be affected by position bias, verbosity bias, and self-enhancement bias. These studies provide the methodological basis for external evidence retrieval and automated output evaluation, respectively; this study combines the two in a system for factual verification.

D. Efficient Inference

High-throughput inference for large language models requires a sufficient number of requests to be processed concurrently. However, the continuously growing KV cache consumes substantial memory, while memory fragmentation and redundant storage can limit batch size. Kwon et al. [9] proposed PagedAttention to manage the KV cache in pages and built vLLM on this basis. Experiments in the original paper show that this method reduces memory waste in the KV cache and increases inference throughput for large language model serving.

III. SYSTEM OVERVIEW

A. Overall Architecture

Briefline consists of a model-training workflow and a news-application workflow. The training workflow uses CNN/DailyMail and KPTimes to construct multi-task training data and completes model fine-tuning and adapter selection. The application workflow applies the selected model to newly published Guardian articles and sequentially performs content generation, evidence retrieval, output verification, and related-news recommendation.

The two workflows are connected by the adapter selected on the validation set. The training workflow produces and selects model adapters, while the application workflow combines the selected adapter with the base model to process news whose source and time range differ from those of the training corpus.

B. Model Training Workflow

As shown in Fig. 1, the training workflow begins with CNN/DailyMail and KPTimes. The system first identifies samples with overlapping content based on semantic relationships between texts and retains representative training data to reduce corpus redundancy. The two datasets are then converted into a unified multi-task format so that the same model can jointly learn news summarization, topic classification, and keyphrase generation.

During fine-tuning, the system saves multiple candidate adapters at different training stages. After training, the evaluator temporarily merges each candidate adapter with the same base

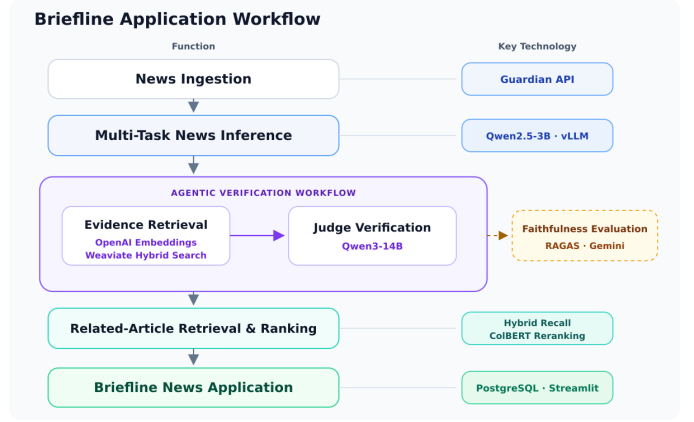


Fig. 2. Briefline application workflow.

model, compares the base model and all candidate models under a unified evaluation setting, and selects the final adapter according to the combined score on the full validation set. The selected adapter is the final output of the training workflow and is loaded with the base model for multi-task news inference in the application workflow.

C. Application Workflow

As shown in Fig. 2, the application workflow first collects newly published articles from the Guardian. The Qwen2.5-3B model loaded with the selected adapter then performs multi-task inference on each article and generates the corresponding news-processing outputs.

The generated outputs then enter the agentic verification workflow. The system retrieves relevant evidence from the source article, and an independent Qwen3-14B judge checks whether the generated content is supported by that evidence. To measure the effectiveness of this verification process, the system includes a separate faithfulness-evaluation branch that compares summaries before and after verification and evaluates the faithfulness of the final content to the evidence in the source article. This evaluation does not participate in the judge's verification or correction decisions; it is used only to quantify the effect of the complete agentic verification workflow.

After verification, the system retrieves and ranks other articles related to the current article, and then provides the news content, verification results, and recommendation relationships to the news frontend. PostgreSQL and Weaviate serve different roles in this process. PostgreSQL stores articles, processing states, and outputs from each stage, connecting the backend workflow to the frontend application. Weaviate stores vectorized content and supports evidence retrieval and related-news matching. The Streamlit frontend does not run models directly; it reads processed data from PostgreSQL.

IV. METHODOLOGY

A. Graph-Based Semantic Data Selection

1) *Selection Objective and Semantic Representation:* The original training sets of CNN/DailyMail and KPTimes contain

more than 500,000 news samples in total. If all data were used directly for fine-tuning, many semantically similar articles would repeatedly contribute to parameter updates, substantially increasing the number of training steps, training time, and total GPU-hours. Briefline therefore performs semantic data selection before model training. It identifies semantic structure in the corpus and retains representative samples from different semantic regions to reduce the training set while preserving as much of the original topic coverage and content diversity as possible.

The system uses `intfloat/e5-base-v2` to convert news articles into dense semantic vectors. E5 learns general-purpose text representations through weakly supervised contrastive learning and is suitable for tasks such as retrieval, classification, and clustering [10]. The system applies mean pooling to the output of the final encoder layer and performs L2 normalization on the resulting vectors, allowing cosine similarity to measure semantic proximity between articles.

2) *Efficient Semantic Graph Construction*: For N samples, exact computation of all pairwise similarities requires processing $O(N^2)$ sample pairs and is unsuitable for the current data scale. Briefline therefore uses a Hierarchical Navigable Small World (HNSW) index to approximately retrieve the semantic neighbors of each sample.

HNSW organizes vectors into a hierarchical proximity graph. Search begins in an upper-level graph with fewer nodes to locate a candidate region quickly and then descends through the hierarchy to refine the results in denser local graphs. This hierarchical search avoids comparing each sample with every other sample and is the main reason Briefline uses HNSW. The original paper shows that the hierarchical structure has favorable complexity scaling and is suitable for large-scale nearest-neighbor retrieval [3].

After candidate neighbors are obtained, the system constructs a weighted semantic graph based on cosine similarity. Each node represents a news sample, and each edge weight represents the semantic similarity between two samples. The system sets a threshold according to the similarity distribution of candidate neighbors and retains only sample pairs whose similarity reaches the threshold and that are mutual neighbors. The similarity threshold filters weak connections, while the mutual-neighbor constraint reduces one-way and unstable adjacency relationships. Together, they convert the candidate neighbors returned by HNSW into a sparse semantic graph of manageable size.

3) *Leiden-Based Semantic Community Detection*: The semantic graph generated by HNSW describes local proximity relationships among samples. To identify more complete semantic structures from these local relationships, the system applies the Leiden algorithm to partition the sparse graph into communities, assigning closely connected articles to the same semantic community.

Leiden does not require the number of communities to be specified in advance, making it suitable for news corpora with an unknown number of topics. The algorithm also guarantees community connectivity and achieved higher runtime efficiency

and partition quality than Louvain in the benchmark and real-world network experiments reported in the original paper [4]. HNSW is therefore used to construct local semantic relationships efficiently, while Leiden partitions those relationships into semantic communities for subsequent representative-sample selection.

4) *Dataset-Specific Clustering Strategies*: CNN/DailyMail and KPTime use the same graph-based semantic clustering framework but use different clustering texts and clustering scopes according to their task structures.

For CNN/DailyMail, the system combines each summary with part of its source article as the clustering text and performs global clustering over the entire training set. The summary usually expresses the central event and main content of the article, so it forms the core of the semantic representation. Part of the source article provides additional context for distinguishing articles with similar summaries but different specific content. After E5 encodes the combined text, the system uses HNSW to construct a unified semantic graph, and Leiden partitions the entire training set into semantic communities.

For KPTime, the system first handles its multi-label and long-tail category structure. The least frequent category associated with each sample is designated as the primary label, so each sample enters only one category group and gives priority to rarer category information. All primary-label groups below the protection threshold are retained, while the remaining groups are clustered separately within each category. The clustering text consists of the title, categories, keyphrases, and part of the source article: the categories and keyphrases provide task-related information, while the article text distinguishes different news content within the same category. Each unprotected group is then independently processed through E5 encoding, HNSW graph construction, and Leiden community detection.

CNN/DailyMail therefore uses summary-centered text representations for global semantic clustering to reduce content redundancy in the summarization corpus. KPTime first protects minority categories and then performs within-category semantic clustering to prevent majority categories from dominating the overall selection results.

5) *Representative Number Allocation*: After semantic clustering, the system determines the number of representative samples according to community size. For a nonempty community of size s , the system supports two sublinear allocation functions, logarithmic and square-root allocation:

$$k_{\log}(s) = \min \{s, k_{\max}, 1 + \lfloor \log_b s \rfloor\} \quad (1)$$

$$k_{\sqrt{}}(s) = \min \left\{ s, k_{\max}, 1 + \left\lfloor \frac{\sqrt{s}}{\tau} \right\rfloor \right\} \quad (2)$$

Here, $k_{\max}$ denotes the maximum number of representatives for a single community, while b and τ control the growth rates of the two functions. At least one sample is retained from every nonempty community, and the actual quota does not exceed either the community size s or the preset upper bound $k_{\max}$.

Briefline ultimately uses square-root allocation. During the experiments, logarithmic allocation was observed to increase the quota for large communities relatively slowly and might not adequately cover their semantic variation. Square-root allocation remains sublinear while assigning relatively more representative samples to large communities. This choice balances corpus compression against semantic coverage of large communities.

KPTimes groups protected as minority categories do not use the preceding quota functions; all samples in these groups are retained. Square-root allocation is applied only to semantic communities formed by Leiden clustering in the remaining groups.

6) *First-Difference-Based Representative Selection*: After community quotas are determined, Briefline selects specific samples using first differences in distance from the centroid. Selecting only samples closest to the community centroid tends to concentrate the results on the dominant semantics of the community while overlooking boundary content and potential subtopics. The method uses changes in the distribution of distances from the centroid to expand topic coverage while retaining central samples, thereby increasing the semantic diversity of the selected training data.

The method is computationally simple: it only requires computing the community centroid, sorting distances from the centroid, and calculating first differences, without another round of pairwise sample comparison or iterative optimization. It is therefore suitable for large-scale corpus selection and allows the model to encounter different news topics even after the training set has been reduced.

Specifically, the system first computes the normalized centroid vector for each community and sorts the samples within the community in ascending order by cosine distance. Let the sorted distances from the centroid be $d_1, d_2, \dots, d_s$, where d_1 and d_s correspond to the samples nearest to and farthest from the community centroid, respectively.

When the community quota is one sample, the system retains the sample nearest to the centroid. When the quota is at least two samples, the system first retains the nearest and farthest samples to cover the dominant semantics and boundary content of the community, respectively. For the remaining quota, the system computes the first differences between adjacent distances from the centroid:

$$\Delta_r = d_{r+1} - d_r, \quad r = 1, \dots, s-1 \quad (3)$$

The system then selects the sample to the right of each distance jump in descending order of Δ_r until the required number of representatives for the community is reached. A large distance jump indicates a marked change in distance from the centroid between adjacent samples and may correspond to a transition between different semantic regions. First differences therefore complement selection based only on central samples and provide additional coverage of boundary content and potential subtopics within a community.

Together with the preceding category-protection, semantic-clustering, and community-quota mechanisms, this method converts a large news corpus into a representative sample set that covers different semantic regions. It reduces the training set while preserving as much of the topic diversity of the original corpus and the minority-category information in KPTimes as possible.

B. Dynamic Multi-Task Fine-Tuning

This study formulates news summarization, news classification, and keyphrase extraction as instruction fine-tuning tasks. CNN/DailyMail is used for the summarization task, while KPTimes is used for joint category and keyphrase generation in the same structured response. Although the three tasks share the news article as input, their output forms and learning difficulty differ substantially. Summarization requires information selection, content compression, and coherent expression, whereas classification and keyphrase extraction produce shorter outputs with stronger structural constraints. A fixed data-mixture ratio and a uniform loss calculation cannot adequately handle this task heterogeneity. This study therefore develops a dynamic multi-task fine-tuning method across four dimensions: data allocation, token supervision, sample normalization, and parameter capacity. The method allows the model's learning emphasis to adjust gradually over the course of training.

1) *Base Model Selection*: News articles typically contain many tokens, and long sequences substantially increase computation and GPU memory use during model training. The base model must therefore provide sufficient contextual modeling capacity while keeping the parameter count manageable to improve the efficiency of long-text fine-tuning and inference.

Based on this tradeoff, this study uses Qwen2.5-3B-Instruct as the base model. It supports long-context modeling, instruction following, and structured output while maintaining a relatively small parameter count, and it can perform summary, category, and keyphrase prediction within a unified generation framework [11]. This balance between capability and efficiency makes the model suitable for multi-task news fine-tuning under limited computational resources.

2) *Training-Stage-Based Dynamic Task Mixture*: A key issue in multi-task fine-tuning is how to allocate the proportions of different training datasets. Mixture-of-Skills (MoS) shows that the training value of different datasets changes with the model's current learning state, making a static data mixture unlikely to remain optimal throughout training [12]. MoS uses an additional scoring network to estimate the utility of different datasets and dynamically adjusts their sampling probabilities. Although this method provides fine-grained data selection based on model state, it also introduces the overhead of training an additional network and alternating optimization.

This study uses training-stage-based task-ratio scheduling as a lightweight alternative. A higher sampling probability is assigned to summarization data at the beginning of training. The proportion of classification and keyphrase data then increases gradually until the two datasets are balanced in the later stages.

Let $p_{\text{sum}}(e)$ and $p_{\text{str}}(e)$ denote the sampling probabilities of summarization data and structured-output data, respectively, at training stage e . The schedule satisfies:

$$p_{\text{sum}}(e) + p_{\text{str}}(e) = 1, \quad p_{\text{sum}}(e) \rightarrow \frac{1}{2} \quad (4)$$

This schedule is driven by the learning characteristics of the tasks. Summarization has longer target sequences and a larger output space and requires the model to learn information selection, semantic compression, and text generation. Classification and keyphrase extraction provide more concentrated supervision and more readily develop stable output patterns. If the short structured tasks dominate early training, the model may fit category and keyphrase patterns prematurely and constrain the development of long-text generation capability.

This study therefore gives greater weight to the summarization task at the beginning of training so that the model first develops content modeling and generation capabilities for long news articles. As training progresses, the proportions of classification and keyphrase extraction increase gradually, allowing the model to learn structured prediction on top of its existing generation capability and ultimately optimize the tasks in a balanced manner. This method implements stage-dependent data scheduling without training an additional data-selection network.

3) *Dynamic Prompt-Loss Scheduling*: Conventional instruction fine-tuning typically computes loss only on the response and masks supervision for the prompt. Previous studies show that in long-input, short-output training settings, assigning a small nonzero loss weight to the prompt can reduce overfitting to fixed response patterns and improve generalization to input variations [13], [14]. Weighted Instruction Tuning further indicates that prompts contain task cues and contextual constraints, and jointly optimizing the prompt and response can balance input understanding and answer generation [15].

This characteristic is highly relevant to multi-task news processing. Summaries, categories, and keyphrases are all determined by the news article, so model outputs strongly depend on the input content. Masking prompt loss entirely concentrates direct supervision on the output. Assigning the prompt and response equal weights, however, may allow the large number of input tokens in long articles to dominate overall optimization. Preliminary experiments in this study also showed that retaining a small input-side weight improved validation performance compared with masking prompt loss completely.

Based on this observation, this study proposes dynamic prompt-loss scheduling, in which the prompt weight gradually decays over training. Let s in $[0, 1]$ denote training progress, and let L_p and L_r denote the prompt and response losses, respectively. The overall objective is:

$$L(s) = \lambda_p(s)L_p + \lambda_r L_r \quad (5)$$

Here, λ_r remains constant throughout training, while $\lambda_p(s)$ gradually decays from a larger initial value to a smaller nonzero value.

A relatively high prompt weight is used early in training, allowing the model to use input-side supervision to adapt to news writing style, content structure, and the domain distribution. As the model adapts to the input distribution, the prompt weight continues to decline and optimization shifts toward accurate generation of summaries, categories, and keyphrases. A nonzero lower bound maintains input-side constraints and prevents the model from relying entirely on output patterns. The method follows a scheduling principle similar to learning-rate decay, using training progress to control the relative importance of different supervision signals.

4) *Sample-Mean Loss*: Differences in news-article length further affect each sample's contribution to the training loss. Conventional token-mean loss averages over all valid tokens in a batch, so longer samples with more tokens naturally exert a greater overall influence. This implicitly turns article length into a sample weight and biases the model toward longer texts.

To reduce this length bias, this study uses sample-mean loss. Let the cross-entropy for token t in sample i be $\ell_{i,t}$, with corresponding weight $w_{i,t}$. A weighted average is first computed within each sample:

$$L_i = \frac{\sum_t w_{i,t} \ell_{i,t}}{\sum_t w_{i,t}} \quad (6)$$

The sample losses are then averaged at the batch level:

$$L_{\text{batch}} = \frac{1}{B} \sum_{i=1}^B L_i \quad (7)$$

Here, $w_{i,t}$ for prompt tokens is determined by the dynamic prompt-loss schedule, while response tokens use a fixed weight. The method first removes within-sample scale differences caused by token count and then assigns equal weight to different samples, preventing long articles from dominating the training loss solely because they contain more tokens.

5) *Adaptive Parameter Allocation with AdaLoRA*: Standard LoRA typically assigns the same rank to different layers and linear mappings, effectively assuming that different weight matrices have similar importance for downstream tasks. In a multi-task setting, this uniform allocation may not use the limited trainable parameters efficiently. Summarization depends more on long-range semantic modeling and text generation, while classification and keyphrase extraction place greater emphasis on discriminative information and local semantic representations. The tasks therefore have different requirements for the model's internal parameters.

AdaLoRA represents low-rank updates in a form that approximates singular value decomposition and dynamically adjusts the rank budget according to the importance of parameter updates [2]. Weight matrices with greater importance retain more effective ranks, while less important updates are progressively compressed. This allows the limited adaptation parameters to concentrate on model components that contribute more to the multi-task objective instead of being allocated uniformly across all weight matrices.

This adaptive parameter allocation matches the multi-task structure of this study: different layers and mappings can receive different adaptation capacities according to the joint training objective, supporting long-text generation, category prediction, and keyphrase extraction under a fixed parameter budget.

C. Validation-Based Model Evaluation and vLLM Inference

1) *Validation-Based Checkpoint Selection*: This study selects the best model checkpoint using the full validation set. The base model and all fine-tuned checkpoints generate summaries, categories, and keyphrases using the same validation data, prompt templates, structured-output format, and deterministic decoding settings, ensuring a consistent basis for comparison across checkpoints.

This study treats each dataset as a primary evaluation unit. CNN/DailyMail corresponds to the summarization task, while KPTimeS corresponds to both category classification and keyphrase generation. If the three tasks were averaged with equal weight, KPTimeS would account for two-thirds of the combined score because it contains two output tasks, causing checkpoint ranking to reflect structured generation on that dataset more heavily.

The experiments further showed that the scores for category classification and keyphrase generation were generally much higher than the summarization score, which would further increase the contribution of KPTimeS to the combined metric. This study therefore uses a two-level aggregation scheme with equal weights across datasets and equal task weights within each dataset.

First, the KPTimeS validation score is computed as the equal-weighted average of the semantic scores for category classification and keyphrase generation:

$$S_{\text{KPTimeS}} = \frac{S_{\text{category}} + S_{\text{keyword}}}{2} \quad (8)$$

The KPTimeS validation score and the CNN/DailyMail summarization score are then aggregated with equal weight:

$$S_{\text{combined}} = \frac{S_{\text{summary}} + S_{\text{KPTimeS}}}{2} \quad (9)$$

After this two-level aggregation, CNN/DailyMail and KPTimeS each account for 50% of the final validation score. Across the three individual tasks, the effective weights for summarization, category classification, and keyphrase generation are 50%, 25%, and 25%, respectively. This design balances the combined metric at the dataset level while reflecting both long-text summarization and structured information generation.

After training, all saved candidate checkpoints are evaluated on the full validation set and ranked according to the preceding aggregated score. The checkpoint with the highest validation score is selected as the final model for evaluation on the held-out test set and for subsequent inference.

2) *Unified Semantic Evaluation*: Summaries, categories, and keyphrases have different output forms, yet each can contain expressions that differ on the surface while being fully or partially equivalent in meaning. Summaries allow

paraphrases and changes in word order; keyphrases may differ in inflection, order, or level of detail; and news categories may have hierarchical relationships and semantic overlap.

Discrete-label tasks commonly use exact match or label-level F1. These metrics treat different labels as independent symbols. When a news sample is close to two related categories, a semantically reasonable prediction with a different label name may still be marked incorrect. Metrics based on lexical overlap for keyphrases and summaries also have difficulty recognizing synonymous expressions, morphological variation, and semantic similarity.

To apply a consistent evaluation principle across the three subtasks, this study represents summaries, categories, and keyphrases as text sequences and uses a continuous semantic score to measure the proximity between a prediction and its reference. Semantically consistent content receives a high score, partially consistent content receives a corresponding partial score, and content unsupported by the reference semantics increases the distance between the two. Based on this principle, the next section develops a unified semantic evaluation metric for all three subtasks.

3) *MoverScore-like Optimal Transport Metric*: Among semantic evaluation methods for text generation, BERTScore uses pretrained language models to produce contextualized token representations, computes the cosine similarity between predicted and reference tokens, and selects the most similar counterpart for each token through local greedy matching [16]. Contextualized representations can recognize synonyms, paraphrases, and changes in word order, making them more suitable than strict lexical matching for evaluating generated outputs that may have multiple valid forms.

The local maximum-matching strategy in BERTScore allows multiple predicted tokens to select the same reference token or the same small set of reference tokens. As long as additional generated content is topically similar to the reference, these tokens may still receive high local similarity scores and inflate the overall score. For the category and keyphrase generation tasks in this study, this characteristic can weaken the metric's ability to detect topically related redundant content.

To reduce the excessive influence of topical similarity on the score, this study first adjusts the token representation by adding embedding-level information to the final-layer contextualized representation, so that the score reflects both high-level semantics and specific lexical forms. Transformer probing studies show that different network layers encode language information at different levels: lower layers mainly retain surface and local information, middle layers represent more syntactic structure, and higher layers gradually form complex semantics [17], [18]. MoverScore also combines representations across layers to form contextualized text representations [19]. Accordingly, this study combines the embedding-level representation and final-layer contextualized representation from RoBERTa and compares local-matching results before and after this combination.

In Table I, the predicted keyphrases include Franklin McMahon and Emmett Till, which are directly related to the reference, as well as expanded expressions not listed in

TABLE I
EFFECT OF COMBINING EMBEDDING-LEVEL AND FINAL-LAYER REPRESENTATIONS ON THE LOCAL-MATCHING SCORE

Item	Content
Reference Categories	US
Predicted Categories	Journalism; Art History; Civil Rights Movement
Reference Keyphrases	McMahon Franklin; News and News Media; Art; Deaths (Obituaries); Newspapers; Magazines; Till Emmett
Predicted Keyphrases	Franklin McMahon; Historical Scene Drawing; Stroke; Emmett Till Trial; Political Conventions; Watercolor Painting; Court Reporting; Freelance Illustrator; Social Justice Documentation; American Legacy Storytelling
Original Keyphrase BERTScore F1	0.8051
Keyphrase BERTScore F1 (Combined Representation)	0.7433

the reference, such as Historical Scene Drawing, Watercolor Painting, and Social Justice Documentation. When only the final-layer representation is used, the keyphrase BERTScore F1 is 0.8051; after combining the embedding-level representation with the final-layer representation, the score decreases to 0.7433. This result shows that adding lexical and local-form information enables the metric to distinguish topical relevance from agreement in specific content more clearly, thereby increasing its sensitivity to differences in output scope.

This representation combination changes only the similarity calculation between tokens; the local maximum-matching strategy remains unchanged. Each predicted token still independently selects the most similar reference token, and multiple predicted tokens can still reuse a small amount of reference semantics. Representation-level combination can therefore improve semantic representation but cannot structurally constrain repeated expansion of topically related content.

To further address this problem, this study introduces the optimal transport approach used in MoverScore. MoverScore represents the predicted and reference texts as semantic distributions with mass weights and uses Earth Mover’s Distance to solve for the minimum transport cost between the two distributions. Local maximum matching independently identifies the best counterpart for each token, whereas optimal transport jointly optimizes the complete transport matrix under marginal-mass constraints, requiring all semantic mass on the prediction and reference sides to participate in the matching process [19].

This study uses the combined RoBERTa token representation as the semantic representation and assigns uniform mass to the prediction and reference sides to construct a MoverScore-like optimal transport metric. Suppose the predicted sequence and reference sequence contain m and n scored tokens, respectively. The token masses on the two sides are defined as:

$$a_i = \frac{1}{m}, \quad b_j = \frac{1}{n} \quad (10)$$

The transport cost between predicted token i and reference token j is defined by the cosine distance between their combined representations:

$$c_{ij} = 1 - \cos(h_i^p, h_j^r) \quad (11)$$

The optimal transport distance is defined as:

$$D_{OT} = \min_{T \geq 0} \sum_i \sum_j T_{ij} c_{ij} \quad (12)$$

subject to the marginal-mass constraints on both sides:

$$\sum_j T_{ij} = a_i, \quad \sum_i T_{ij} = b_j \quad (13)$$

The final semantic score is:

$$S_{OT} = 1 - D_{OT} \quad (14)$$

Here, T_{ij} denotes the mass transported from predicted token i to reference token j . All mass on the prediction side must enter the transport matrix, while the total mass received by each reference token is constrained by b_j . A single reference token therefore cannot be reused without restriction by multiple predicted tokens.

When a prediction contains additional content, the new tokens must also participate in global transport. Content consistent with the reference can form shorter transport paths, whereas unsupported content must be transported to semantically more distant reference tokens, increasing the overall transport cost. This mechanism allows the score to reflect both information shared by the prediction and reference and information added to or omitted from the prediction relative to the reference.

The reference in Table II contains three keyphrases, while the prediction contains 26. J. R. Smith and New York Knicks correspond directly to the reference, whereas the remaining content includes many players and teams not listed in the reference. Because these entities are all related to basketball, BERTScore F1 after combining the embedding-level and final-layer representations still reaches 0.7806. This shows that representation-level combination does not eliminate the tolerance of local maximum matching for topically related redundant content.

In MoverScore-like OT, all predicted mass participates in the global transport process. Content supported by the reference forms lower-cost transport paths, while the remaining entities increase the overall transport cost, reducing the score for the same sample to 0.6814, an absolute difference of 0.0992. This result preserves the semantic contribution of correctly predicted content while also reflecting the overall deviation caused by the large number of additional entities.

TABLE II
COMPARISON OF BERTSCORE-STYLE MATCHING AND MOVERSCORE-LIKE OT ON THE SAME SAMPLE

Item	Content
Reference Categories	Basketball; Sports
Predicted Categories	Sports; NBA; Player Misconduct
Reference Keyphrases	Basketball; Knicks; J. R. Smith
Predicted Keyphrases	J. R. Smith; Fine; Shoe Trick; New York Knicks; Dennis Rodman; Michael Jordan; Rudy Tomjanovich; Chris Paul; Stephen Curry; LeBron James; Kevin Durant; Russell Westbrook; Kobe Bryant; Steph Curry; Golden State Warriors; Los Angeles Lakers; Chicago Bulls; Boston Celtics; Toronto Raptors; Philadelphia 76ers; Milwaukee Bucks; Cleveland Cavaliers; Indiana Pacers; Atlanta Hawks; Charlotte Hornets; Orlando Magic
Keyphrase BERTScore F1 (Combined Representation)	0.7806
MoverScore-like OT	0.6814

Based on the preceding analysis, this study ultimately uses a MoverScore-like optimal transport metric that combines embedding-level and final-layer representations. Contextualized representations identify synonymous expressions and semantic similarity, lower-layer information improves recognition of differences in specific content, and global mass constraints further limit repeated reuse of reference semantics. The same semantic-distance calculation is then applied uniformly to summaries, categories, and keyphrases, providing a consistent evaluation basis for checkpoint selection.

Fine-tuning loss reflects the model's ability to predict target tokens conditionally, whereas downstream task evaluation concerns the sequence-level semantic quality of complete generated outputs, creating a loss-evaluation mismatch [20]. This study therefore does not save an adapter based on the lowest loss. Instead, each staged evaluation uses a fixed set of 300 validation samples per task to compute MoverScore-like OT, and the four highest-ranked candidate adapters are retained according to the dataset-balanced combined score.

4) *Efficient Validation Inference with vLLM*: Evaluating the full validation set requires the base model and multiple fine-tuned checkpoints to generate summaries, categories, and keyphrases separately. Inference therefore involves many fixed task prefixes, long news inputs, and variable-length outputs. This study uses vLLM for this validation inference. As a high-throughput inference engine for large language models, vLLM improves batch inference efficiency through cache reuse, paged KV-cache management, and dynamic request scheduling.

Automatic prefix caching. Automatic prefix caching stores previously computed token prefixes and their KV-cache blocks. When a later request has an identical prefix, vLLM can reuse the corresponding cache directly and avoid repeated prefill computation for the shared portion. In this study, samples for the same checkpoint and task share the system prompt, task instructions, and structured-output specification; the main variation is the news article that follows. The fixed task template can therefore be reused throughout validation of one checkpoint. Because different checkpoints use different model parameters, their KV caches are built separately.

Chunked prefill and PagedAttention. Chunked prefill divides long inputs into smaller prefill chunks and schedules them together with active decoding requests. vLLM V1 first sched-

ules pending decode requests and then uses the remaining token budget for prefill; long inputs that cannot fit fully within the current budget are automatically divided into chunks. This scheduling method combines compute-bound prefill with memory-bound decode more effectively.

PagedAttention divides the KV cache of each request into fixed-size physical blocks and uses a block table to map logical sequences to noncontiguous GPU memory. Cache space can therefore be allocated on demand as a sequence grows, reducing memory fragmentation and redundant cache copies [9]. The news articles in this study are long, and both input and generation lengths vary substantially across samples. Chunked prefill prevents long inputs from blocking other requests, while PagedAttention improves GPU-memory utilization for the dynamic KV cache. Experiments by Kwon et al. [9] show that, at the same latency level, vLLM achieves approximately two to four times the throughput of FasterTransformer and Orca, with larger gains for long sequences, larger models, and complex decoding scenarios.

Continuous batching. Continuous batching updates the active request set at each scheduling iteration. Requests that have finished generation leave the active batch and release computation and cache resources; unfinished requests continue decoding; and waiting requests enter subsequent iterations as resources permit. Summaries, categories, and keyphrases have different output lengths, and samples within the same task also produce EOS at different times. This mechanism allows shorter outputs to leave immediately after completion rather than waiting for the longest sequence in a static batch, reducing resource use by completed requests.

Asynchronous output processing and CUDA Graph. Asynchronous output processing overlaps CPU-side output handling with subsequent model execution, reducing GPU stalls caused by output conversion and request-state updates. CUDA Graph captures and replays repeated sequences of GPU operations, reducing CPU overhead from Python scheduling and kernel launches. During full-validation-set inference, many requests repeat the same generation and output-processing workflow. These two mechanisms further reduce CPU synchronization waits and the overhead of repeatedly submitting operations.

These mechanisms act at different stages of validation inference: automatic prefix caching reduces repeated prefill

for shared task templates; chunked prefill and PagedAttention handle long inputs and dynamic KV-cache requirements; continuous batching schedules variable-length outputs; and asynchronous output processing with CUDA Graph reduces CPU-side execution overhead. Together, they improve generation efficiency for the base model and multiple fine-tuned checkpoints on the full validation set.

V. APPLICATION

Although model training in this study is based on historical news from CNN/DailyMail and KPTimes, the goal is to apply the model to current news processing so that the system can handle articles published after the training period. The application stage therefore does not reuse test-set samples and instead uses recently published news.

However, the original sources of CNN/DailyMail and KPTimes do not provide interfaces suitable for free and continuous collection of current full-text articles in this study. The Guardian Open Platform provides an official Content API that allows noncommercial research to access article text within specified rate and daily limits. This study therefore collects Guardian articles incrementally and applies the trained model to these current articles to generate summaries, categories, and keyphrases.

A. Agentic Workflow and Faithfulness Evaluation

1) *Motivation for Agentic Verification*: The trained model can generate outputs directly, but it cannot guarantee that all content is supported by the source article. This study therefore adds an independent judge after the generation model to verify summaries, categories, and keyphrases.

Previous studies have demonstrated the effectiveness of post-generation verification. RARR, proposed by Gao et al. [21], retrieves external evidence for model outputs, identifies conflicts between outputs and evidence through consistency judgments, and revises content only when an inconsistency is found while preserving as much of the original output as possible. Chain-of-Verification, proposed by Dhuliawala et al. [22], organizes an initial answer, verification-question planning, independent answers to those questions, and a final answer into consecutive stages. Their experimental results show that staged verification can reduce hallucinations across several types of tasks. These studies support the evidence-based post-generation checking and staged verification used in this work.

Building on this work, the present study further considers the inference cost of verifying long news articles. The most direct approach is to have the judge read the full article for every generated output, determine whether revision is needed, and produce a corrected output when necessary. News articles are usually long, so applying full-article verification to every sample would substantially increase inference time. In fact, many original outputs require no revision. This study therefore uses a two-stage workflow: Stage 1 uses shorter evidence to determine whether an output can be retained directly, while Stage 2 reads the full article only for outputs that remain uncertain after the initial screening. This conditional routing

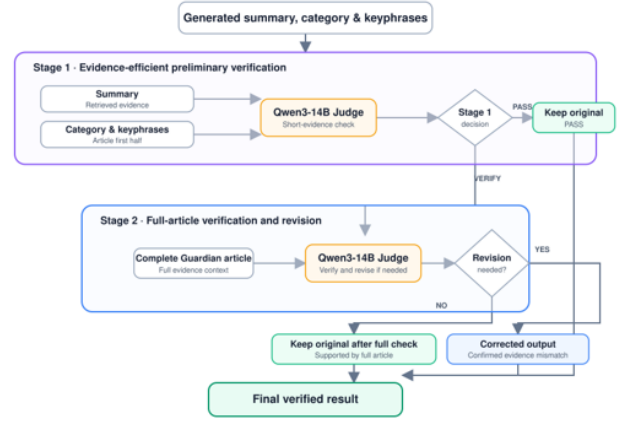


Fig. 3. Two-stage agentic verification workflow.

limits costly full-article verification to questionable outputs while retaining the benefits of post-generation verification, and it is the core of the agentic workflow in this study.

The unified judge in this system is Qwen3-14B with thinking mode enabled. The verification task requires comparing generated outputs with news evidence and determining whether an output can be retained directly, requires confirmation from the full article, or truly needs revision. Qwen3's thinking mode is designed for complex multi-step reasoning tasks [23], so it is used for both stages of judgment in this study.

The two-stage agentic verification workflow is shown in Fig. 3.

2) *Stage 1: Evidence-efficient Preliminary Verification*: Stage 1 aims to identify outputs that can be retained directly using shorter evidence. Because summaries, categories, and keyphrases require different evidence for judgment, this study designs separate initial-screening methods and submits the resulting requests together to the judge.

Retrieval-augmented summary verification. This study applies RAG to summary verification [6]. The system first divides each generated summary into verifiable factual statements and splits the corresponding Guardian article into sentence-centered text chunks. Adjacent chunks retain contextual overlap so that cross-sentence references to people, times, and event relationships remain available in the retrieval representation.

The system uses OpenAI text-embedding-3-small to generate vector representations of factual statements and source-article chunks and stores the source vectors in Weaviate. Weaviate builds an HFresh index for these vectors to accelerate semantic similarity retrieval. During retrieval, Weaviate combines keyword matching with vector similarity and filters by article identifier, ensuring that evidence for each factual statement can be retrieved only from its corresponding source article. The judge therefore reads only the generated summary and its retrieved evidence in the first stage, rather than the full article.

If the retrieved evidence supports every statement in the summary, the judge outputs PASS and the system retains the original summary. If a statement lacks support or still

requires confirmation from the full article, the judge outputs VERIFY. Stage 1 performs screening only and does not revise the summary.

Joint category and keyphrase verification. Categories and keyphrases primarily describe the news topic, key people, and events, and the first half of an article usually contains the main information required for an initial judgment. The system therefore places the generated categories, generated keyphrases, and the first half of the article in a single request for joint evaluation by the judge. If both fields are consistent with the article, the request passes directly. If uncertainty remains, the system marks the categories, keyphrases, or both for Stage 2 verification.

vLLM inference. Summary-screening requests and category-keyphrase screening requests are submitted together to vLLM and scheduled through continuous batching. Categories and keyphrases do not require two separate inference calls, and summary requests can enter the same batch-processing workflow, improving overall inference efficiency in Stage 1.

3) *Stage 2: Full-article Verification and Revision:* Only outputs marked VERIFY in Stage 1 enter Stage 2. For summaries, the judge rechecks the original summary against the full Guardian article. For categories and keyphrases, the system rechecks only the fields marked in Stage 1 and likewise provides the full article.

Stage 2 performs full-article verification and produces the final output in a single call. The judge changes the original output only when the full article shows that a revision is truly needed. If the full article supports the original answer, the system retains it. Entering Stage 2 therefore indicates that further confirmation is required, not that the output is necessarily incorrect.

4) *Routing Results and Inference Cost:* Table III summarizes the two-stage routing results for Guardian articles. Summary verification included 2,500 initial-screening requests: 1,765 passed directly, and 735 entered full-article verification. Joint category-keyphrase verification included 2,499 initial-screening requests: 1,814 passed directly, and 685 entered full-article verification.

Of all 4,999 initial-screening requests, 71.6% did not enter Stage 2, and only 28.4% required the full article. This result shows that most outputs could be evaluated using retrieved evidence or the first half of the article, avoiding repeated full-article verification for all samples.

Entering Stage 2 does not necessarily mean that a revision is required. Among the 735 summaries that entered full-article verification, 607 were ultimately changed, while 128 retained the original summary after the full article was read. This shows that Stage 1 can route outputs with insufficient evidence to full-article confirmation while avoiding unnecessary revisions when the original output is correct.

5) *Faithfulness Evaluation:* The preceding routing results show that the two-stage workflow can reduce full-article verification. It is also necessary to determine whether the workflow actually improves output quality. Summaries compress and reorganize multiple facts from the source article

and are more likely than categories or keyphrases to generate unsupported content. Through human evaluation, Maynez et al. [24] found that several abstractive summarization models produce unfaithful information to varying degrees. This study therefore uses summaries to evaluate whether the agentic workflow improves the faithfulness of generated outputs. The evaluation compares summaries before and after verification and uses the full Guardian article as the evidence in both cases.

This study uses the RAGAS Faithfulness metric, with Gemini 2.5 Flash-Lite performing its semantic judgments. RAGAS first decomposes a summary into independent statements and then determines whether each statement can be inferred from the source article [25]. If a summary contains N statements and N_s of them are supported by the article, Faithfulness is defined as:

$$F = \frac{N_s}{N} \quad (15)$$

This study also uses the complementary value as a derived hallucination score:

$$H = 1 - F \quad (16)$$

B. Related News Recommendation

The related-news recommendation function helps readers discover other reports related to the topic or event of the current article. This function must consider both retrieval efficiency and result relevance. Fast retrieval is suitable for finding candidate articles in a news collection but may not distinguish accurately between reports with similar topics and different specific events. A finer-grained relevance model provides a fuller semantic comparison but is unsuitable for processing every article in the collection individually. Based on this tradeoff, the study uses a retrieval-reranking pipeline common in information retrieval, sequentially narrowing the topic scope, retrieving candidate articles, and reranking the candidates.

1) *Topic-aware Hybrid Retrieval:* Candidate retrieval first controls the search scope. A news collection contains different topics such as politics, business, sports, and culture. If the system searches all articles directly, topically irrelevant articles may enter the candidate set because of local word overlap or general semantic similarity. The system therefore first identifies related category combinations based on the category information of the current article and limits subsequent article retrieval to those categories. Narrowing the topic scope does not directly determine the final recommendations; instead, it provides a more focused search space for candidate retrieval and reduces interference from cross-topic candidates in later ranking.

Within the related-category scope, the system uses Weaviate hybrid search to retrieve candidate articles. Related articles may share people, organizations, locations, or event names, or they may describe similar content using different wording. Keyword retrieval alone can miss semantically related articles with different surface forms, while dense-vector retrieval alone may weaken exact cues such as proper names. Wang

TABLE III
STAGE 1 SCREENING AND STAGE 2 ROUTING RESULTS

Verification Target	Screening Requests	Passed Directly	Entered Full-Article Verification	Full-Article Verification Skip Rate
Summary	2500	1765	735	70.6%
Categories and Keyphrases	2499	1814	685	72.6%
Total	4999	3579	1420	71.6%

TABLE IV
STAGE 2 FULL-ARTICLE SUMMARY VERIFICATION RESULTS

Full-Article Verification Result	Number of Summaries	Share of Summaries Entering Stage 2
Original Summary Revised	607	82.6%
Original Summary Retained	128	17.4%
Total	735	100.0%

et al. [26] found that lexical and dense retrieval provide complementary relevance signals and that combining them can improve candidate retrieval. This study therefore uses hybrid search to consider both keyword matching and vector-based semantic similarity, forming a candidate set at relatively low cost.

The main purpose of this stage is to recall as many potentially relevant articles as possible rather than to determine the final order directly. The system retrieves up to 50 candidate articles from the narrowed topic scope and passes them to a finer-grained relevance model.

2) *ColBERT-based Reranking*: Hybrid search is suitable for narrowing the candidate set quickly, but its ranking mainly depends on lexical matching and overall semantic representations. When multiple articles discuss similar topics, final recommendations require further comparison of specific people, events, and their semantic relationships. This study therefore uses ColBERTv2 to rerank the candidate articles retrieved by hybrid search.

A conventional bi-encoder usually compresses the query and document into separate single vectors, providing efficient retrieval but potentially losing some token-level information. A cross-encoder can provide rich text interaction through joint encoding, but it must perform inference separately for each query-document pair and is difficult to apply directly to the entire news collection. ColBERT uses late interaction: it encodes queries and documents separately while retaining token-level vectors and computes fine-grained similarity between them during matching. Experiments by Khattab and Zaharia [7] show that ColBERT achieves competitive ranking performance while substantially reducing the query cost of BERT-style joint ranking. This makes it suitable as a reranking model after candidate retrieval.

The ColBERTv2 model used in this study further improves the retrieval quality and space efficiency of late-interaction representations [27]. The system does not use ColBERT to search the entire news collection; it reranks only the candidates

returned by hybrid search and ultimately returns the top 10 articles. Low-cost topic constraints and hybrid search therefore narrow the candidate scope, while ColBERTv2 performs fine-grained relevance assessment within the candidate set.

C. Persistent State Management and Frontend Integration

Briefline applies the trained model to a continuously updated stream of Guardian articles. Model inference, output verification, and quality evaluation depend on a GPU or external APIs, so the application system must preserve processing results across runs to avoid repeated computation. This section describes how PostgreSQL, run records, and evaluation checkpoints support incremental processing and recovery after interruption, and how persisted results are provided to an independently running frontend.

1) *Incremental Processing and Stage Recovery*: Incremental processing concentrates computing resources on new articles and unfinished work from previous runs. PostgreSQL stores outputs from each stage and uses idempotent writes to avoid creating duplicate records. Before each run, the system determines the processing state of every article from existing results and assigns only new or incomplete articles to the appropriate stage. An article advances to a later stage only after the current stage is complete.

In addition to database state, the system generates a run record for every pipeline execution. The record stores the set of articles included in the run and the progress of each stage. After an interruption, the system restores the original processing scope from the run record and then uses results already stored in PostgreSQL to determine which stages still need to be executed.

Run records and database state jointly support recovery across runs: the run record preserves the scope of an execution, while the database provides the outputs that have actually been completed. The system can therefore continue unfinished processing and reuse existing inference and verification results, reducing repeated GPU computation and external API calls.

2) *Faithfulness Evaluation Checkpointing*: In addition to the application pipeline, Faithfulness Evaluation is a long-running external evaluation process. Large-scale evaluation takes substantial time and may be affected by API quotas, rate limits, or temporary connection failures. This stage therefore requires a progress-saving mechanism independent of the main application data.

This study saves the result immediately after evaluating the original-final summary pair for each article. When the evaluation is rerun, the system reuses existing complete scores

and continues from articles that remain unfinished. If the judge did not revise a summary, the original score is also used as the final score, avoiding repeated evaluation of identical content.

Per-sample checkpointing separates evaluation progress from a single run, allowing completed results to be reused across runs and limiting the failure of an individual sample to a local scope. Faithfulness Evaluation can therefore resume after an interruption without reevaluating all articles.

3) *Broad-Category Taxonomy and Read-Only Frontend*: The fine-grained categories confirmed by the judge describe specific news topics, but their large number makes them unsuitable as the primary navigation structure of the frontend. To create a stable and easy-to-browse news taxonomy, the system must retain the original category information while organizing it into a limited number of broad categories.

This study aggregates the final generated categories and uses Gemini to produce 8 to 14 broad categories. Every category must be preserved in full and assigned to exactly one broad category. The resulting category mapping is then stored in PostgreSQL.

The Streamlit frontend reads persisted article results and category mappings directly and uses them to build the home and category pages. Model generation, judge verification, and category organization are all performed in the backend; page access does not require loading the generation model, judge, or retrieval model. This separation allows the compute-intensive backend to update data continuously while the read-only frontend runs independently in a standard CPU environment.

VI. EXPERIMENTAL RESULTS

This section evaluates Briefline in four areas: training-data selection, multi-task model performance, agentic verification, and evaluation efficiency. The experiments measure the reduction in training-corpus size produced by semantic data selection, the task-performance gains from multi-task fine-tuning, the improvement in factual faithfulness of news summaries from agentic verification, and the acceleration of the full evaluation workflow provided by vLLM.

A. Overview of Key Results

Table V summarizes the main experimental results of Briefline. Graph-based semantic data selection compresses the training corpus from 547,036 to 72,026 samples. Multi-task fine-tuning increases the average score across the three tasks from 0.7178 to 0.8164. Agentic verification increases the RAGAS Faithfulness of news summaries from 0.9166 to 0.9510, corresponding to a 41.2% reduction in the derived hallucination score. With vLLM, the full evaluation workflow runs more than three times faster.

Overall, the results show that Briefline substantially reduces the training corpus while improving performance across the three target tasks and uses agentic verification to increase consistency between generated news summaries and the source evidence. vLLM further shortens the full post-training evaluation time and improves the efficiency of model iteration.

TABLE V
SUMMARY OF KEY BRIEFLINE RESULTS

Evaluation Item	Baseline	Briefline	Change
Training Corpus Size	547,036	72,026	−86.83%
Three-Task Average on the Held-Out Test Set	0.7178	0.8164	+13.74%
RAGAS Faithfulness	0.9166	0.9510	+3.44 pp
Derived Hallucination Score	0.0834	0.0490	−41.2%
Full Evaluation Runtime	Transformers	vLLM	More than 3× Speedup

TABLE VI
GRAPH-BASED SEMANTIC DATA SELECTION RESULTS

Dataset	Original Samples	Selected Samples	Retention Rate	Reduction Rate
CNN/DailyMail	287,113	37,739	13.14%	86.86%
KPTimes	259,923	34,287	13.19%	86.81%

B. Graph-Based Semantic Data Selection

Graph-based semantic data selection is applied to the CNN/DailyMail and KPTimes training sets. The selection process uses `intfloat/e5-base-v2` to generate text vectors and uses a fixed random seed of 42. The selected samples jointly form the training corpus used for subsequent multi-task model fine-tuning.

CNN/DailyMail is reduced from 287,113 to 37,739 samples, with a retention rate of 13.14%. KPTimes is reduced from 259,923 to 34,287 samples, with a retention rate of 13.19%. The retention rates of the two datasets differ by only 0.05 percentage points, indicating that semantic data selection produces a balanced compression effect across the two training sources.

The selected training corpus contains 72,026 samples, an 86.83% reduction from the original 547,036 samples. This result demonstrates the effectiveness of graph-based semantic selection in constructing a compact training set, allowing the model to complete subsequent adaptation with a smaller and more representative corpus.

C. Multi-Task Model Performance

The multi-task model uses Qwen/Qwen2.5-3B-Instruct as its base model and is fine-tuned on the selected 37,739 CNN/DailyMail samples and 34,287 KPTimes samples. The model is trained for six epochs with a batch size of 8 and a learning rate of 4×10^{-4} . During training, AdaLoRA dynamically adjusts the adaptation rank from 128 to 90, retaining approximately 70% of the initial rank. The model has 239.5 million trainable parameters, representing 7.20% of all parameters.

The model jointly learns three target tasks: CNN/DailyMail summarization, KPTimes category classification, and KPTimes keyphrase generation. Evaluation uses 1,000 validation samples and 1,000 held-out test samples from each of CNN/DailyMail

TABLE VII
PERFORMANCE OF THE BASE AND MULTI-TASK FINE-TUNED MODELS ON THE THREE TARGET TASKS

Task	Dataset	Base Model	Fine-Tuned Model	Abs. Change (pp)	Relative Change
Category Classification	KPTimes	68.19%	87.35%	+19.16	+28.09%
Keyphrase Generation	KPTimes	70.79%	80.00%	+9.21	+13.01%
Summary Generation	CNN/DailyMail	76.36%	77.58%	+1.22	+1.60%
Three-Task Average	All Tasks	71.78%	81.64%	+9.86	+13.74%

TABLE VIII
EFFECT OF AGENTIC VERIFICATION ON THE FACTUAL FAITHFULNESS OF NEWS SUMMARIES

Summary Version	Faithfulness	Derived Hallucination Score
Original Generated Summary	0.9166	0.0834
Final Summary from the Judge	0.9510	0.0490
Change	+3.44 pp	−41.2%

and KPTimes. CNN/DailyMail samples evaluate summarization, while KPTimes samples evaluate both category classification and keyphrase generation.

The base and fine-tuned models use the same generation configuration, including greedy decoding and a maximum of 128 new tokens. Outputs for all three tasks are evaluated using a RoBERTa-large-based MoverScore semantic similarity metric. Model checkpoints are selected according to validation-set performance, and the held-out test set is used to report final performance.

The fine-tuned model improves on all three target tasks. The gains are larger for category classification and keyphrase generation. The improvement in summarization is smaller, mainly because the instruction-tuned base model already has strong general-purpose summarization capability, with an initial score of 76.36%, leaving relatively limited room for further improvement.

D. Agentic Verification and Factual Faithfulness

The agentic-verification experiment evaluates the effect of the judge stage on evidence consistency in generated news summaries. The experiment includes 2,492 valid comparison samples from Guardian articles. Each sample contains the original generated summary and the final summary processed by the judge, with the corresponding source article used as the evidence context.

The experiment uses Gemini 2.5 Flash-Lite as the RAGAS evaluation model. The derived hallucination score is calculated as:

$$S_{\text{hallucination}} = 1 - S_{\text{faithfulness}} \quad (17)$$

After judge verification, the Faithfulness of the final news summaries increases from 0.9166 to 0.9510, an absolute increase of 3.44 percentage points. The corresponding derived hallucination score decreases from 0.0834 to 0.0490, a relative reduction of 41.2%.

The experimental results show that the judge stage substantially increases consistency between the final news summaries and the source evidence. Agentic verification can correct evidence-related deviations before the summaries enter the final Briefline output, improving the factual quality of the generated content.

E. End-to-End Evaluation Efficiency

The end-to-end efficiency experiment measures the actual runtime of the full evaluation workflow after model training. Hugging Face Transformers and vLLM are used separately to perform the same model-generation and evaluation tasks. Both evaluation paths use the same trained model, data, tasks, and evaluation conditions and run on the same NVIDIA A800 80 GB GPU.

vLLM completes the full evaluation workflow more than three times faster than Transformers. This result covers model generation and metric computation across the complete test workflow, showing that optimized batch generation can substantially shorten the model-evaluation cycle.

The evaluation speedup directly improves the experimental iteration efficiency of Briefline, allowing generation, scoring, and result aggregation for post-training model checkpoints to be completed more quickly. With the model, data, tasks, and evaluation conditions held constant, vLLM provides a substantial improvement in runtime efficiency for the full evaluation workflow.

VII. CONCLUSION

This paper designs and implements Briefline to address data scale, multi-task processing, factual reliability, and the transition to practical use in news-intelligence systems. The system connects training-data construction, model adaptation, current-news processing, evidence retrieval, agentic verification, and news recommendation into a complete workflow, moving model capability beyond benchmark evaluation and applying it to continuously updated real-world news content.

The main contribution of Briefline is an end-to-end technical pipeline from representative training data to reliable news outputs. Graph-based semantic data selection constructs a compact training corpus; multi-task fine-tuning enables one model to perform summarization, category classification, and keyphrase generation; and hybrid retrieval with agentic verification provides evidence support and factual correction for generated content. Together, these components form an end-to-end system for news processing.

The experimental results show that a selected compact corpus can effectively support multi-task model adaptation, agentic verification can increase consistency between news summaries and source evidence, and an optimized evaluation workflow can improve model-iteration efficiency. Overall, Briefline demonstrates a feasible path from model training on benchmark data to a real-world news-intelligence application and effectively connects data efficiency, task capability, factual reliability, and system runtime efficiency.

REFERENCES

- [1] H. W. Chung et al., “Scaling Instruction-Finetuned Language Models,” *Journal of Machine Learning Research*, vol. 25, no. 70, pp. 1–53, 2024.
- [2] Q. Zhang et al., “AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning,” in *International Conference on Learning Representations (ICLR)*, 2023. <https://openreview.net/forum?id=lq62uWRJjiY>
- [3] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020. <https://doi.org/10.1109/TPAMI.2018.2889473>
- [4] V. A. Traag, L. Waltman, and N. J. van Eck, “From Louvain to Leiden: Guaranteeing Well-Connected Communities,” *Scientific Reports*, vol. 9, art. 5233, 2019. <https://doi.org/10.1038/s41598-019-41695-z>
- [5] A. K. M. Abbas et al., “SemDeDup: Data-Efficient Learning at Web-Scale Through Semantic Deduplication,” *ICLR 2023 Workshop on Mathematical and Empirical Understanding of Foundation Models*, 2023. arXiv:2303.09540.
- [6] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [7] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” in *Proceedings of SIGIR*, pp. 39–48, 2020. <https://doi.org/10.1145/3397271.3401075>
- [8] L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems*, vol. 36, pp. 46595–46623, 2023.
- [9] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, pp. 611–626, 2023. <https://doi.org/10.1145/3600006.3613165>
- [10] L. Wang et al., “Text Embeddings by Weakly-Supervised Contrastive Pre-training,” arXiv:2212.03533, 2022.
- [11] Qwen Team, “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024.
- [12] M. Wu, T.-T. Vu, L. Qu, and G. Haffari, “Mixture-of-Skills: Learning to Optimize Data Usage for Fine-Tuning Large Language Models,” in *Proceedings of EMNLP*, pp. 14226–14240, 2024.
- [13] M. Huerta-Enochian and S. Y. Ko, “Instruction Fine-Tuning: Does Prompt Loss Matter?” in *Proceedings of EMNLP*, pp. 22771–22795, 2024.
- [14] Z. Shi et al., “Instruction Tuning With Loss Over Instructions,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [15] A. Chatterjee et al., “On the Effect of Instruction Tuning Loss on Generalization,” *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 1360–1380, 2025.
- [16] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating Text Generation with BERT,” in *International Conference on Learning Representations (ICLR)*, 2020. <https://openreview.net/forum?id=SkeHuCVFDr>
- [17] G. Jawahar, B. Sagot, and D. Seddah, “What Does BERT Learn about the Structure of Language?” in *Proceedings of ACL*, pp. 3651–3657, 2019. <https://doi.org/10.18653/v1/P19-1356>
- [18] I. Tenney, D. Das, and E. Pavlick, “BERT Rediscovered the Classical NLP Pipeline,” in *Proceedings of ACL*, pp. 4593–4601, 2019. <https://doi.org/10.18653/v1/P19-1452>
- [19] W. Zhao et al., “MoverScore: Text Generation Evaluating with Contextualized Embeddings and Earth Mover Distance,” in *Proceedings of EMNLP-IJCNLP*, pp. 563–578, 2019. <https://doi.org/10.18653/v1/D19-1053>
- [20] S. Wiseman and A. M. Rush, “Sequence-to-Sequence Learning as Beam-Search Optimization,” in *Proceedings of EMNLP*, pp. 1296–1306, 2016. <https://doi.org/10.18653/v1/D16-1137>
- [21] L. Gao et al., “RARR: Researching and Revising What Language Models Say, Using Language Models,” in *Proceedings of ACL*, pp. 16477–16508, 2023. <https://doi.org/10.18653/v1/2023.acl-long.910>
- [22] S. Dhuliawala et al., “Chain-of-Verification Reduces Hallucination in Large Language Models,” in *Findings of ACL*, pp. 3563–3578, 2024. <https://doi.org/10.18653/v1/2024.findings-acl.212>
- [23] A. Yang et al., “Qwen3 Technical Report,” arXiv:2505.09388, 2025.
- [24] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On Faithfulness and Factuality in Abstractive Summarization,” in *Proceedings of ACL*, pp. 1906–1919, 2020. <https://doi.org/10.18653/v1/2020.acl-main.173>
- [25] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, “RAGAS: Automated Evaluation of Retrieval Augmented Generation,” in *Proceedings of the EACL System Demonstrations*, pp. 150–158, 2024. <https://doi.org/10.18653/v1/2024.eacl-demo.16>
- [26] S. Wang, S. Zhuang, and G. Zucco, “BERT-Based Dense Retrievers Require Interpolation with BM25 for Effective Passage Retrieval,” in *Proceedings of ICTIR*, pp. 317–324, 2021. <https://doi.org/10.1145/3471158.3472233>
- [27] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, “ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction,” in *Proceedings of NAACL*, pp. 3715–3734, 2022. <https://doi.org/10.18653/v1/2022.naacl-main.272>